Module 1: Python Foundations

Reference Document for AI Students

An academic reference covering the core concepts of Python:
variables, control flow, data structures, functions, and advanced topics.

Comprehensive Lecture Notes

May 11, 2026

1 Introduction

This module provides a thorough introduction to Python for students entering artificial intelligence and data science. Python has become the lingua franca of modern AI owing to its readability, extensive libraries, and gentle learning curve. No prior programming experience is assumed. The document proceeds from fundamental building blocks (variables, types, operators) through control flow and data structures, to functions and advanced topics. Every concept is accompanied by a code example, and comparison tables are provided for rapid reference.

2 Variables, Types, and Operators

2.1 Dynamic Typing

Python is **dynamically typed**: the *value* carries the type, not the variable. A single variable can hold values of different types over its lifetime. The built-in `type()` function reveals the type of any expression.

```
x = 42                # x is an int
print(type(x))        # <class 'int'>
x = "hello"           # x is now a str
print(type(x))        # <class 'str'>
x = [1, 2, 3]         # x is now a list
print(type(x))        # <class 'list'>
```

2.2 The Four Fundamental Types

1. **Integers (`int`)** — Whole numbers of arbitrary precision. Python integers are unbounded and grow to accommodate any magnitude. Example: `age = 25`, `huge = 2**1000`.
2. **Floating-point numbers (`float`)** — Fractional numbers using IEEE 754 double-precision (64-bit). Example: `pi = 3.14159`, `sci = 1.5e-3`.
3. **Strings (`str`)** — Immutable sequences of Unicode characters, delimited by single, double, or triple quotes. Example: `name = "Ada"`, `multi = """line1\nline2"""`.
4. **Booleans (`bool`)** — Logical values `True` and `False`. Booleans are a subclass of `int`, so `True == 1` and `False == 0`.

```
student_count = 47              # int
average_grade = 82.5            # float
course_name   = "AI Principles" # str
is_passed     = True            # bool
```

2.3 Operator Precedence

Python evaluates expressions in descending precedence order. When operators of equal precedence appear, evaluation proceeds left to right.

Prec.	Operator	Description
1 (highest)	`**`	Exponentiation
2	`+x, -x, ~x`	Unary plus, minus, bitwise NOT
3	`*, /, //, %`	Multiply, divide, floor divide, modulus
4	`+, -`	Addition, subtraction
5	`«, »`	Bitwise shift
6	`&`	Bitwise AND
7	`^`	Bitwise XOR
8	`|`	Bitwise OR
9	`==, !=, >, <, >=, <=, is, in`	Comparisons, membership
10	`not`	Logical NOT
11	`and`	Logical AND
12 (lowest)	`or`	Logical OR

Table 1: Operator precedence, highest to lowest.

2.4 Type Conversion

Python supports **implicit** conversion (e.g., `int` promoted to `float` in mixed expressions) and **explicit** conversion via `int()`, `float()`, `str()`, and `bool()`. Critical gotchas: `int()` truncates toward zero (does *not* round), and invalid conversions raise `ValueError`.

```
result = 3 + 4.5           # 7.5 (implicit int -> float)
number = int("123")        # 123
text   = str(3.14)         # "3.14"

print(int(3.9))            # 3 (truncation, not rounding!)
print(int(-3.9))           # -3
# int("abc")               # ValueError
```

3 Control Flow

Control flow structures determine the order of statement execution. Python provides conditional branching (`if`/`elif`/`else`), definite iteration (`for` loops), and indefinite iteration (`while` loops). Code blocks are delimited by indentation (typically four spaces).

3.1 Conditional Statements: `if`/`elif`/`else`

A conditional evaluates a Boolean expression and executes the block corresponding to the *first* true condition. If no condition is true, the `else` block (if present) executes. Only one branch runs per chain, even if multiple conditions would otherwise match.

```
grade = 87
if grade >= 90:
    print("Excellent")
elif grade >= 75:
    print("Good")
elif grade >= 60:
    print("Satisfactory")
else:
    print("Needs Improvement")
# Output: Good
```

3.2 `for` Loops, `range()`, and `enumerate()`

Python's `for` loop iterates over any iterable object. The `range(start, stop, step)` function generates integer sequences, with `stop` being exclusive. The `enumerate(iterable, start)` function yields `(index, value)` pairs, eliminating the need for manual counter variables.

```
for i in range(1, 6):
    print(f"Iteration {i}")

for i in range(0, 10, 2):
    print(i)                    # 0, 2, 4, 6, 8

subjects = ["Math", "Physics", "AI"]
for idx, subject in enumerate(subjects, start=1):
    print(f"{idx}: {subject}")  # 1: Math, 2: Physics, 3: AI
```

3.3 `while` Loops, `break`, and `continue`

A `while` loop repeats while its condition is true. `break` immediately exits the innermost loop; `continue` skips the remainder of the current iteration and proceeds to the next.

```
count = 0
while count < 5:
    print(count)
    count += 1

for num in range(1, 20):
    if num % 7 == 0:
        print(f"Multiple of 7: {num}")  # 7
        break

for num in range(1, 11):
    if num % 2 == 0:
        continue
    print(num)                          # 1,3,5,7,9
```

3.4 Logical Operators

Python's logical operators (`and`, `or`, `not`) use **short-circuit evaluation**: evaluation stops as soon as the result is determined. This can be exploited to guard expensive or potentially failing

expressions.

Op	Example	Short-Circuit Behaviour
`and`	`a > 0 and b > 0`	If left is false, right is not evaluated
`or`	`a > 0 or b > 0`	If left is true, right is not evaluated
`not`	`not (a > b)`	Single operand; no short-circuit

Table 2: Logical operators with short-circuit evaluation.

4 Data Structures

Python's built-in data structures are implemented in C for performance and provide clean, intuitive interfaces. Mastery of lists, tuples, sets, and dictionaries is indispensable for data-intensive work.

4.1 Lists

A list is an **ordered**, **mutable** sequence defined with square brackets. Elements are accessed by zero-based index; negative indices count from the end. Slicing uses `list[start:stop:step]` with exclusive `stop`.

```
numbers = [10, 20, 30, 40, 50]
print(numbers[0])       # 10       (first)
print(numbers[-1])      # 50       (last)
print(numbers[1:4])     # [20,30,40]
print(numbers[::-1])    # [50,40,30,20,10]  (reversed)

fruits = ["apple", "banana"]
fruits.append("cherry")           # ["apple","banana","cherry"]
fruits.insert(1, "blueberry")     # ["apple","blueberry","banana","cherry"]
fruits.remove("banana")           # ["apple","blueberry","cherry"]
popped = fruits.pop()             # "cherry" (removes and returns last)
fruits.sort()                     # ["apple","blueberry"]
```

4.2 Tuples

A tuple is an **ordered**, **immutable** sequence defined with parentheses (or without, when unambiguous). Tuples are preferred when data should not change after creation, when the fixed structure conveys meaning (e.g., coordinates), or when a slight performance gain is desirable.

```
point  = (3, 7)
person = ("Ada", 28, "Engineer")
single = (42,)            # trailing comma for singleton

# Unpacking
name, age, role = person
a, b = 10, 20
a, b = b, a               # elegant swap: a=20, b=10

# point[0] = 5            # TypeError: immutable!
```

4.3 Sets

A set is an **unordered** collection of **unique** elements, defined with curly braces or `set()`. Sets are mutable but their elements must be hashable. Membership testing is $O(1)$ on average, making sets ideal for deduplication and fast lookups.

```
evens = {2, 4, 6, 8, 10}
uniq  = set([1, 2, 2, 3, 3, 4])   # {1, 2, 3, 4}

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B)    # {1,2,3,4,5,6}   union
print(A & B)    # {3,4}             intersection
print(A - B)    # {1,2}             difference
print(A ^ B)    # {1,2,5,6}         symmetric difference
```

4.4 Dictionaries

A dictionary is an **insertion-ordered** (Python 3.7+) collection of **key-value** pairs. Keys must be hashable; values may be of any type. Use `get()` for safe key access to avoid `KeyError`.

```python
student = {"name": "Ada", "age": 25, "courses": ["AI", "Math"]}
print(student["name"])                # Ada
print(student.get("grade", "N/A"))    # N/A (safe default)

student["grade"] = "A"                # add new key
del student["age"]                    # remove key

for key, value in student.items():
    print(f"{key} -> {value}")
```

4.5 Structure Comparison

Property	List	Tuple	Set	Dict
Ordered	Yes	Yes	No	Yes (3.7+)
Mutable	Yes	No	Yes	Yes (values)
Allows duplicates	Yes	Yes	No	No (keys)
Indexed by	Integer	Integer	N/A	Key
Syntax	`[ ]`	`(,)`	`{ }`	`{:}`
Membership speed	$O(n)$	$O(n)$	$O(1)$	$O(1)$ (keys)

Table 3: Comparison of Python's four primary built-in data structures.

5 Strings and Functions

5.1 String Methods

Strings are immutable; transformation methods return *new* strings. The `join()` method is called on the *separator*—an inverted but deliberate design.

```python
text = "  Hello, Python World!  "
print(text.upper())           # "  HELLO, PYTHON WORLD!  "
print(text.strip())           # "Hello, Python World!"
print(text.find("Python"))    # 9  (index, -1 if absent)
print(text.count("o"))        # 3
print("World" in text)        # True

words = text.strip().split()  # ['Hello,', 'Python', 'World!']
print(" - ".join(words))      # "Hello, - Python - World!"

print(text.replace("World", "AI"))  # "  Hello, Python AI!  "
```

5.2 Formatted String Literals (f-strings)

F-strings (Python 3.6+) embed expressions inside string literals: `f"{expression}"`. They support precision, alignment, padding, and multi-line strings via triple quotes. F-strings are faster than `%-formatting` and `str.format()`.

```python
name, score, courses = "Ada", 92.5678, 3
print(f"Student {name} scored {score:.2f}")   # 92.57
print(f"Pad left:  {name:>10}")               # "       Ada"
print(f"Pad right: {name:<10}")               # "Ada       "
print(f"Next year: {courses + 1} courses")

# Multi-line f-string
report = f"""
  Name:   {name}
  Score:  {score:.1f}%
  Status: {"Pass" if score >= 60 else "Fail"}
"""
```

5.3 Function Definition

Functions encapsulate reusable code blocks using `def`. They accept parameters, may return values, and should include a docstring. Without a `return` statement, a function implicitly returns `None`.

```python
def greet(name, greeting="Hello"):
    """Return a greeting string."""
    return f"{greeting}, {name}!"

print(greet("Ada"))                          # Hello, Ada!
print(greet("Bob", greeting="Hi"))           # Hi, Bob!
print(greet(greeting="Welcome", name="Charlie")) # Welcome, Charlie!
```

5.4 Parameter Categories

Python supports five parameter categories: **positional** (matched by order), **keyword** (matched by name), **default** (optional with fallback), ***args** (excess positional into a tuple), and ****kwargs** (excess keyword into a dict).

```
def describe(name, age, *hobbies, **details):
    print(f"{name}, age {age}")
    print(f"Hobbies: {', '.join(hobbies)}")
    for key, value in details.items():
        print(f"  {key}: {value}")

describe("Ada", 28, "reading", "cycling",
         city="London", occupation="Scientist")
# Ada, age 28
# Hobbies: reading, cycling
#   city: London
#   occupation: Scientist
```

5.5 Scope: The LEGB Rule

Variable resolution follows **LEGB**: **L**ocal, **E**nclosing (nonlocal), **G**lobal, **B**uilt-in. Variables defined in a function are local and destroyed on return. Use `global` to modify a global variable from within a function.

```
counter = 0    # global
def increment():
    global counter
    counter += 1
    msg = "done"    # local
increment()
print(counter)      # 1
# print(msg)        # NameError: not defined outside function
```

6 Advanced Topics

6.1 List Comprehensions

A list comprehension applies an expression to each element of an iterable, with optional filtering. The syntax is `[expr for var in iterable if cond]`. Dictionary and set comprehensions follow analogous patterns.

```
squares = [x**2 for x in range(1, 6)]           # [1,4,9,16,25]
evens   = [x**2 for x in range(1,11) if x%2==0] # [4,16,36,64,100]

# Flatten a 2D list
matrix = [[1,2,3], [4,5,6], [7,8,9]]
flat   = [n for row in matrix for n in row]     # [1..9]

# Dict comprehension
names = ["Ada", "Bob", "Charlie"]
lens  = {n: len(n) for n in names}              # {'Ada':3, 'Bob':3, ...}

# Set comprehension
uniq_sq = {x**2 for x in [1,2,2,3,3,4]}        # {1,4,9,16}
```

While elegant, deeply nested comprehensions harm readability. If a comprehension spans more than two lines, consider a traditional loop instead.

6.2 Lambda Functions, `map()`, and `filter()`

A **lambda** is an anonymous, single-expression function: `lambda args: expr`. Lambdas are commonly used with `map()` (apply a function to each element) and `filter()` (retain elements for which the function returns truthy). In modern Python, list comprehensions are often preferred for readability.

```
square = lambda x: x**2
print(square(5))                                # 25

nums = [1, 2, 3, 4, 5]
print(list(map(lambda x: x*2, nums)))           # [2,4,6,8,10]
print(list(filter(lambda x: x%2==0, nums)))     # [2,4]

# Comprehensions (often preferred)
doubled = [x*2 for x in nums]
evens   = [x for x in nums if x%2==0]
```

6.3 Common Errors

Recognising common exception types accelerates debugging. The four most frequent errors in beginner code are:

- `NameError` — A variable or function name is used before being defined. Common cause: misspelling or referencing a variable outside its scope.
- `TypeError` — An operation is applied to an object of inappropriate type (e.g., `"Score: " + 95`). Fix with explicit type conversion.
- `IndexError` — A sequence is accessed with an index out of range (e.g., `[10,20,30][5]`). Guard with `len()` checks.
- `ValueError` — A function receives a value of correct type but inappropriate content (e.g., `int("abc")`). Handle with `try`/`except`.

```
# TypeError fix
score = 95
msg = "Score: " + str(score)

# IndexError guard
items = [10, 20, 30]
item = items[5] if len(items) > 5 else "out of range"

# ValueError handling
try:
    num = int("abc")
except ValueError:
    print("Invalid conversion")
```

7 Summary and Key Takeaways

This module has covered the foundational concepts of Python programming. The following checklist enumerates every topic for self-assessment.

1. **Dynamic Typing** — type belongs to value, not variable; `type()` inspects.
2. **Fundamental Types** — `int`, `float`, `str`, `bool`.
3. **Operator Precedence** — fixed order; parentheses clarify intent.
4. **Type Conversion** — implicit (`int`→`float`) and explicit (`int()`, `str()`, etc.).
5. **Conditionals** — `if`/`elif`/`else`, first true branch executes.
6. **`for` Loops** — iterate iterables; `range()` and `enumerate()`.
7. **`while` Loops** — `break` exits early; `continue` skips.
8. **Logical Operators** — `and`, `or`, `not` with short-circuit.
9. **Lists** — ordered, mutable, indexed; `append`, `insert`, `pop`, `sort`.
10. **Tuples** — ordered, immutable; unpacking; elegant swaps.
11. **Sets** — unordered unique elements; $O(1)$ membership; union, intersection, difference.
12. **Dictionaries** — key-value, $O(1)$ lookup; `get()` for safe access.
13. **String Methods** — `strip`, `split`, `join`, `find`, `replace`.
14. **F-strings** — `f"{expr}"` with precision, alignment, padding.
15. **Functions** — `def` with positional, keyword, default, `*args`, `**kwargs`.
16. **LEGB Scope** — Local → Enclosing → Global → Built-in.
17. **Comprehensions** — `[expr for v in iter if c]` for lists, dicts, sets.
18. **Lambda**/`map`/`filter` — anonymous functions; higher-order operations.
19. **Common Errors** — `NameError`, `TypeError`, `IndexError`, `ValueError`.

Quick Reference of Important Syntax Patterns

Mastery of these concepts prepares the student for object-oriented programming, file handling, and Python's extensive AI and data science libraries.

Pattern	Example
Variable	`x = 42`
Type check	`type(x)`
If-elif-else	`if x > 0: ... elif x < 0: ... else: ...`
For-range	`for i in range(10):`
For-enumerate	`for idx, val in enumerate(items, 1):`
While	`while condition:`
List	`lst = [1, 2, 3]`
List comp.	`[x**2 for x in range(5) if x > 0]`
Tuple unpack	`a, b = (1, 2)` / `a, b = b, a`
Set ops	`A | B, A & B, A - B, A ^ B`
Dict access	`d.get("key", default)`
F-string	`f"{n} scored {s:.2f}"`
Function	`def fn(a, b=0, *args, **kwargs):`
Lambda	`lambda x: x + 1`
Map/Filter	`map(fn, iter), filter(fn, iter)`
Try-except	`try: ... except Exc: ...`

Table 4: Quick-reference syntax patterns.